Computer Assignment2
Q2_ report

I. Why Kafka is used as the ingestion layer instead of writing directly into a relational database

Apache Kafka is a better ingestion layer for real-time food orders because it is designed for **high-throughput, low-latency event streaming**, whereas relational databases are primarily designed for **structured storage, querying, and transactions**.

Performance

Kafka is optimized for handling a very large number of incoming events continuously. It writes data sequentially to disk as an append-only log, which makes ingestion very fast.

A relational database, in contrast, usually performs heavier work per write, such as indexing, transaction handling, constraint checking, and locking. This makes it less suitable as the first entry point for a massive stream of real-time events.

Scalability

Kafka scales horizontally by distributing data across partitions and brokers. As order volume grows, more brokers and partitions can be added to handle higher throughput.

Relational databases can scale, but write-heavy scaling is harder and often more expensive. Many relational systems scale vertically first, and horizontal scaling for high write loads is more complex.

Data access patterns

Kafka is built for **streaming access patterns**. Producers append events, and consumers read them sequentially. It is ideal when many downstream services need the same stream of events.

Relational databases are built for **query-based access patterns**, where users retrieve data using SQL, filters, joins, and updates. They are excellent for structured lookup and reporting, but not as efficient as a streaming backbone.

Suitability for real-time streaming

Kafka is specifically designed for real-time pipelines. It decouples producers and consumers, allows multiple consumers to independently process the same event, and supports durable event retention. This makes it ideal for use cases like validation, analytics, monitoring, fraud checks, and notifications running in parallel.

Computer Assignment2
Q2_ report

A relational database is not naturally designed to act as a high-speed event bus for many independent consumers.

One case where a relational database is better

A relational database is still the better choice when the main requirement is **transactional consistency and complex querying**.

For example, storing the final confirmed orders, payments, and customer account balances is better handled by a relational database because these operations require ACID guarantees, updates, and reliable structured queries.

II. Stateless vs stateful processing, and why stateless processing is preferred at ingestion

Difference between stateless and stateful processing

Stateless processing means each message is processed independently, using only the data inside that message. The consumer does not need memory of previous events or information from external systems.

Example: checking whether a phone number starts with `+91` or `080`.

Stateful processing means processing depends on stored context, previous events, or external data sources such as a database or cache.

Example: checking whether a user has already placed five orders today, or whether a restaurant is currently active in a database.

Why stateless processing is preferred at the ingestion layer

Stateless processing is preferred because it is:

- **faster**, since no database lookup is needed
- **simpler**, because each event is self-contained
- **more reliable**, because there are fewer external dependencies
- **easier to scale**, since any consumer instance can process any message independently
- **lower latency**, which is important for ingestion pipelines

At the ingestion layer, the goal is to quickly accept, validate, and route incoming events without slowing down the pipeline.

Computer Assignment2
Q2_ report

Risks if Kafka consumers become state-dependent

If Kafka consumers depend on external state, several problems can appear:

1. Increased latency

Each message may require a database or API call, slowing down processing.

2. Reduced throughput

The consumer can only process messages as fast as the external system responds.

3. More failures

If the database or cache becomes unavailable, the consumer may fail or stop processing messages.

4. Harder scaling

Consumers become less independent because they rely on shared external state, which can become a bottleneck.

5. Inconsistent results

If the external state changes during processing, replaying the same Kafka message later may produce a different result, which reduces predictability.

6. Operational complexity

State management introduces issues such as synchronization, retries, race conditions, and recovery handling. For these reasons, stateless validation is ideal at the ingestion stage.

III. Kafka as a distributed commit log rather than a traditional message queue

Calling Kafka a **distributed commit log** means that Kafka stores messages as an ordered, append-only sequence of records distributed across brokers and partitions. Messages are not simply removed after one consumer reads them. Instead, Kafka keeps them for a retention period, and consumers track their own reading position using offsets.

Computer Assignment2
Q2_ report

What this means in practice

When a producer sends an order event to Kafka, the message is appended to a log. Each message gets an offset, which identifies its position in the log. Consumers read messages by offset and decide for themselves how far they have processed.

This is different from a traditional message queue, where a message is often removed once consumed by one consumer.

How this enables message replay

Because messages remain in Kafka for a configured retention period, a consumer can go back and read old messages again by resetting its offsets.

This is useful for:

- debugging
- rebuilding application state
- reprocessing data after fixing a bug
- running new analytics on historical events

How this enables multiple independent consumers

Different consumer groups can read the same topic independently, each with its own offsets.

For example:

- one consumer group validates orders
- another calculates analytics
- another sends notifications

All of them can consume the same `ashpaz.order` topic without interfering with each other.

How this enables fault tolerance

Kafka is distributed across multiple brokers, and topic partitions can be replicated. If one broker fails, another replica can continue serving the data. Also, because consumers track offsets separately, a failed consumer can restart and continue from its last committed offset instead of losing all progress.

Summary

Kafka behaves like a durable distributed log of events rather than a simple one-time delivery queue. This design provides:

Computer Assignment2
Q2_ report

- durable storage of events
- replay capability
- independent parallel consumers
- strong fault tolerance through replication and offset management

error sample:

{"order_id": "d9d78cea-e910-4007-a3d0-8eb3975f9b98", "error_type": "INVALID_PHONE", "error_reason": ["INVALID_PHONE"], "original_order": {"order_id": "d9d78cea-e910-4007-a3d0-8eb3975f9b98", "user_id": "user_000364", "restaurant_id": 8739, "restaurant_name": "Durbari Restaurant", "restaurant_city": "Jayanagar", "cuisines": ["North Indian", "Chinese", "Mughlai"], "phone_number": "0711290371", "request_online": false, "request_table": true, "order_time": "2026-05-29T06:45:22.354558Z", "items": [{"food_item": "Tandoori Chicken", "category": "North Indian", "unit_price": 151.93, "quantity": 1}, {"food_item": "Malai Tikka", "category": "Mughlai", "unit_price": 197.48, "quantity": 1}], "order_price": 349.41}}

{"order_id": "0a84a66e-8639-4597-b412-c09c20ad579c", "error_type": "INVALID_PHONE", "error_reason": ["INVALID_PHONE"], "original_order": {"order_id": "0a84a66e-8639-4597-b412-c09c20ad579c", "user_id": "user_000287", "restaurant_id": 10541, "restaurant_name": "Pulimunchi", "restaurant_city": "Malleshwaram", "cuisines": ["Mangalorean", "Seafood"], "phone_number": "++91139223658", "request_online": true, "request_table": false, "order_time": "2026-05-29T06:46:37.285012Z", "items": [{"food_item": "Chicken Lollipop", "category": "Mangalorean", "unit_price": 182.78, "quantity": 1}, {"food_item": "Biryani", "category": "Seafood", "unit_price": 225.48, "quantity": 1}], "order_price": 408.26}}

{"order_id": "8d5a7c11-5e2a-4462-ad8b-db4d360d5a1b", "error_type": "ORDER_MODE_CONFLICT", "error_reason": ["ORDER_MODE_CONFLICT"], "original_order": {"order_id": "8d5a7c11-5e2a-4462-ad8b-db4d360d5a1b", "user_id": "user_000134", "restaurant_id": 8892, "restaurant_name": "The Yellow Submarine", "restaurant_city": "JP Nagar", "cuisines": ["Continental", "North Indian", "Chinese", "Pizza", "Thai"], "phone_number": "+918722147555", "request_online": true, "request_table": true, "order_time": "2026-05-29T05:27:08.354456Z", "items": [{"food_item": "Maggi", "category": "Thai", "unit_price": 434.44, "quantity": 1}, {"food_item": "Bbq Chicken Pizza", "category": "Pizza", "unit_price": 289.36, "quantity": 1}, {"food_item": "Buttermilk", "category": "Chinese", "unit_price": 254.62, "quantity": 1}], "order_price": 978.42}}

{"order_id": "9f2d26de-a470-4c67-94b3-35999cf787b4", "error_type": "PRICE_MISMATCH", "error_reason": ["PRICE_MISMATCH"], "original_order": {"order_id": "9f2d26de-a470-4c67-94b3-35999cf787b4", "user_id": "user_000279", "restaurant_id": 11849, "restaurant_name": "Lassi Shop", "restaurant_city": "Residency Road", "cuisines": ["Beverages", "Ice Cream"], "phone_number": "+917411165592", "request_online": true, "request_table": false, "order_time": "2026-05-29T06:44:27.445239Z", "items": [{"food_item": "Paan Ice Cream", "category": "Ice Cream", "unit_price": 68.49, "quantity": 1}], "order_price": 37.31}}